

-4 / +31 lines of 180 · 4 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (15 code line(s) added, 4 removed) · restructuring level 0

BEFORE	AFTER	file
131 #endif	131 #endif	
132 }	132 }	
133	133	
134 static real_t kernel_s341(void)	134 static real_t kernel_s341(void)	
135 {	135 {	
136 int j;		
137 for (int nl = 0; nl < R; nl++) {	136 for (int nl = 0; nl < R; nl++) {	
138 j = -1;	137 /* Pass 1 (parallel): Count matching elements where b[i] > 0 */	
	138 int count = 0;	
	139 #pragma omp parallel for reduction(+:count)	
	140 for (int i = 0; i < LEN_1D; i++) {	
	141 if (b[i] > (real_t)0.) count++;	
	142 }	
	143	
	144 /* Allocate temporary buffer to hold matched elements */	
	145 real_t* temp = (real_t*)malloc(count * sizeof(real_t));	
	146	
	147 /* Pass 2 (parallel): Gather matching elements to temp buffer	
	148 * write_pos is shared and incremented atomically via critical section	
	149 * to assign each thread a unique write index. Data write happens outs	
	150 * critical section to allow true parallelism. */	
	151 int write_pos = 0;	
	152 #pragma omp parallel for shared(write_pos)	
139 for (int i = 0; i < LEN_1D; i++) {	153 for (int i = 0; i < LEN_1D; i++) {	
140 if (b[i] > (real_t)0.) {	154 if (b[i] > (real_t)0.) {	
141 j++;	155 int idx;	
142 a[j] = b[i];	156 /* Critical section only protects index allocation, not data w	
	157 #pragma omp critical	
	158 {	
	159 idx = write_pos++;	
	160 }	
	161 temp[idx] = b[i];	
	162 }	
	163 }	
	164	
	165 /* Pass 3 (parallel): Copy compacted results back to array a */	
	166 #pragma omp parallel for	
	167 for (int i = 0; i < count; i++) {	
	168 a[i] = temp[i];	
	169 }	
	170	
	171 free(temp);	
145 pb_mix(nl);	172 pb_mix(nl);	
146 }	173 }	
147 return (real_t)0;	174 return (real_t)0;	
148 }	175 }	
149	176	